FulfillOps HTTP API contract — version 1.1 (API unchanged; advanced reliability clarification)

Authoritative specification for the graded project. The implementation language is free. The public checker uses only Python's standard library. All paths are relative to http://localhost:8080 during local assessment. JSON uses UTF-8. Request and response payloads must use the field names exactly as specified.

Fixed catalog and seed inventory

KIT-100 — unit_price_cents: **1299**; seed available: **7**

PART-200 — unit_price_cents: **599**; seed available: **12**

PACK-300 — unit_price_cents: **249**; seed available: **5**

Seed a **fresh isolated test database** once; repeated application restarts must not recreate spent stock. Multiple valid orders decrease available stock immediately at acceptance time, **not at worker completion**. Every accepted item reservation is atomic. Neither retries nor crashes can reserve stock twice. quantity must be a positive integer (not boolean), and multiple lines with the same SKU are added together for stock checks and cost calculation.

GET /health/live

Status **200**, content-type JSON:

{"status":"ok"}

Return 200 while the API process is alive, even if the database is unavailable. A dead process has no response.

GET /health/ready

Status **200** when the database is reachable and required migrations have been applied:

{"status":"ready"}

When the service cannot accept orders safely, return **503**:

{"status":"not_ready"}

GET /api/v1/catalog

Status **200**:

{"products":[{"sku":"KIT-100","unit_price_cents":1299},{"sku":"PART-200","unit_price_cents":599},
{"sku":"PACK-300","unit_price_cents":249}]}

Order does not matter. Additional fields are allowed, but all three SKUs/prices are mandatory.

GET /api/v1/inventory/{sku}

Status **200**:

{"sku":"KIT-100","available":7}

available is a nonnegative JSON integer. A non-existent SKU returns **404** with code SKU_NOT_FOUND.

POST /api/v1/orders

Required HTTP header: Idempotency-Key: <unique-client-key>, 8–128 printable ASCII characters. A missing key returns **400** with IDEMPOTENCY_REQUIRED.

Example JSON request:

{"customer_id":"CUST-042","items":[{"sku":"KIT-100","quantity":1},{"sku":"PART-200","quantity":2}]}

Validation: customer_id nonempty string of at most 64 characters. items is a nonempty array of 1–10 objects, with valid SKU and positive integer quantity 1–20. The request body limit is 16 KiB. Reject invalid JSON with **400** and code INVALID_JSON; general structural mistakes with **400** INVALID_REQUEST; invalid quantity with **422** INVALID_QUANTITY; invalid SKU with **422** INVALID_SKU; insufficient stock with **409** OUT_OF_STOCK.

Successful first acceptance is **202**:

{"order_id":"0d9674de-7fac-4c13-bb2f-c4bcbaf743da","status":"queued"}

The response must be emitted **after** durable order creation and stock reservation in one atomic transaction, and before worker processing. The worker is a **separate** process/service, not merely a function invoked synchronously within the API request. order_id is a stable nonempty string.

For identical Idempotency-Key and semantically identical payload, reply **202** with the original order_id and status:"queued" as an *acceptance receipt* even if the order has since completed. Do not reserve stock a second time. A previously used key combined with a different payload returns **409** IDEMPOTENCY_CONFLICT. Normalize JSON key ordering when comparing payloads. The idempotency mapping must survive a restart.

If multiple clients submit simultaneous orders for the last N units of stock, no more than N units may be committed in total. Requests that exceed inventory return **409** OUT_OF_STOCK. The public race check uses distinct client keys and one item per request. No negative stock, overselling or partial allocation is permitted.

GET /api/v1/orders/{order_id}

Status **200**:

{"order_id":"0d9674de-7fac-4c13-bb2f-c4bcbaf743da","status":"completed","total_cents":2497}

Statuses are queued, processing, completed, or failed. total_cents must equal the sum of seeded unit prices multiplied by quantities. A new accepted order should normally become completed within 15 seconds in a healthy local test setup. The persistent order record must survive API or worker restart. Unknown IDs return **404** with code ORDER_NOT_FOUND.

Error object — all errors

{"error":{"code":"OUT_OF_STOCK","message":"Not enough items available"}}

The public checker validates error.code; message must be a safe, nonempty string. Do not include stack traces, credentials, SQL, connection strings or personal data.

GET /metrics

Status **200**, text/plain in Prometheus exposition format. At minimum expose numerical series for:

fulfillops_orders_created_total
fulfillops_orders_failed_total
fulfillops_worker_retries_total
fulfillops_http_requests_total

At least one latency histogram or equivalent numerical measurement is required for an observed p95. The grader may inspect both the text and operational dashboard/queries. Metric labels must not contain customer identifiers or idempotency keys.

Security, recovery and compatibility

API and worker must not rely on process-local variables for authoritative inventory or idempotency state.

Readiness must fail when storage is unavailable; liveness should remain independent of storage.

Do not expose database credentials or admin database reset routes.

Idempotency and stock invariants must hold during worker crashes and concurrent requests.

Provide a safe, documented way to recreate an isolated **test-only** dataset. It must refuse to operate against production.

API local tests are destructive: run the black-box checker against your isolated local **test** environment only.

Advanced operational acceptance contract (9.5/10 edition)

The **HTTP routes, response schemas, seeded products, error codes and existing acceptance timeouts above are unchanged**. These are additional guarantees assessed through a combination of the public --stress checker, independent team tests, controlled demonstrations and instructor review; there are **no newly required public endpoints**.

Simultaneous identical keys across API instances: eight concurrent identical requests must accept exactly one persisted order and reserve only one unit. Every response must give the same order_id; replay after an API restart must remain consistent. Requests with distinct keys compete for stock atomically.

Durable at-least-once work delivery: two worker instances must safely claim jobs; an interrupted job can be recovered without a second business-side effect. A bounded retry policy and stored dead-letter/operator-review state are mandatory. A poisoned order that exhausts retries becomes failed, remains auditable and is not silently deleted or automatically restocked.

Rollout safety: a faulty Kubernetes revision must fail readiness/progression and trigger an intentional recovery to a known-good Git state. Prove observed API availability over a 30-second window during one good rolling update. A manual kubectl patch is not the final GitOps fix.

Operational evidence: demonstrate a measured p95 from at least 200 local request samples, a CI-produced SBOM, an enforced quality/security gate, and a report that distinguishes observed evidence from design intentions.

Test safety: --concurrency and --stress consume inventory; the latter requires an isolated test dataset with **5 PACK-300** units available at start and the normal catalog inventory. Do not run against a production deployment or shared classroom service. Reset scripts must reject production configurations.

Quick demonstration

```
# Liveness
curl -i http://localhost:8080/health/live

# Create an order
curl -i -X POST http://localhost:8080/api/v1/orders \
  -H 'Content-Type: application/json' \
```

```
  -H 'Idempotency-Key: example-unique-001' \
  -d '{"customer_id":"CUST-042","items":[{"sku":"KIT-100","quantity":1}]}'

# Test all public behaviors on an ISOLATED test database
python fulfillops_public_checker.py --base-url http://localhost:8080 --concurrency --stress
```